

Prompt Engineering Cheat Sheet

16 tables · 150 concepts · Updated 2026-09-25 · [cheatgrid.com](#)

Prompt engineering is the practice of writing and refining the instructions that guide large language models (LLMs) toward the output you want. It grew out of transformer-based models like GPT, Claude and Gemini, evolving from simple question-answer patterns into a discipline covering reasoning frameworks, output control and security. As models grow more capable, the field is converging with **context engineering**, the broader practice of shaping all information a model receives. That makes the structure, format and context of a prompt matter as much as the words themselves.

CONTENTS

Core Prompting Approaches 1

Output Control and Formatting 2

Message Roles and Context Structure 3

Sample Selection and Example Design 4

Generation Parameters and Sampling 4

Emotion and Persona Techniques 5

Reasoning and Decomposition Techniques 6

Advanced Reasoning Patterns 7

Prompting for Reasoning Models 8

Prompt Chaining and Workflow Orchestration 9

Prompting Agents and Long-Horizon Tasks 10

Multimodal and Vision-Language Prompting 11

Domain-Specific Applications 11

Optimization and Automation 12

Safety and Robustness 13

Anti-Patterns and Common Pitfalls 13

References 14

TABLE 1

Core Prompting Approaches

Foundational ways to shape an LLM's answer using the wording of a single prompt, before reaching for reasoning chains or tools. They differ mainly in how much you show the model (no examples, one, or several), how you frame the request (persona, situational context, explicit rules), and whether the model is asked to clarify the question for itself first.

Technique	Example	Description
Prompt elements	Instruction: Classify the review. Context: 1 (worst) to 5 (best). Input Data: "Food was okay." Output Indicator: Rating:	• A prompt is built from up to four parts: instruction, context, input data, and an output indicator • not every part is required; the mix depends on the task
Zero-shot prompting	Translate to French: Hello	• Model performs task without examples, relying solely on pre-training knowledge • fast but less reliable for complex or domain-specific tasks
Few-shot prompting	English: cat → French: chat English: dog → French: chien English: bird → ?	• Provides 2–5 example input-output pairs before the query • significantly improves accuracy and consistency for nuanced tasks

Technique	Example	Description
One-shot prompting	Example: "angry" → negative Classify: "delightful" → ?	• Single demonstration example • useful when task is straightforward but model needs format guidance
Role prompting	You are an expert oncologist. Explain CAR-T therapy.	• Assigns a persona or expertise to the model • most effective for controlling tone, style, and output format rather than expanding factual knowledge
Instruction following	List three benefits. Use bullet points. Keep under 50 words. If none apply, reply "None found."	• Explicit directives on what, how, and constraints • a directive can add a fallback branch for input that lacks an expected field, instead of leaving that case undefined
Contextual prompting	Background: User is a beginner. Task: Explain neural networks.	• Provides situational information (audience, constraints, domain) to shape the response appropriately • explaining the reasoning behind an instruction helps the model apply it correctly
Rephrase and Respond (RaR)	Rephrase and expand this question, then answer: Why is the sky blue?	• Model rephrases the question before answering • improves accuracy by resolving ambiguity in the original phrasing

TABLE 2

Output Control and Formatting

These techniques shape what the model returns and how it is structured, so downstream code can parse it and humans can read it. A key distinction runs through the table: prompt-only instructions (asking for JSON, a length, or "do not" rules) are soft requests the model can miss, while API-level features like structured outputs and max_tokens enforce hard constraints.

Technique	Example	Description
Structured output (JSON)	Return as JSON: {"name": str, "age": int}	• Enforces a specific schema (JSON, XML, YAML) • only schema-enforced structured outputs (constrained decoding) guarantee conformance; a prompt-only "Return JSON" can still emit invalid or extra text
XML tag structuring	<context>text</context> <instructions>summarize</instructions>	• Wraps prompt sections in semantic XML tags • reduces ambiguity by marking boundaries; an Anthropic best practice especially effective with Claude, but clear delimiters help most models
Delimiters and sections	### Input text ### Output summary	• Uses markers (### headers, fenced code, ---) to separate sections • reduces ambiguity about what content the model should process vs. generate
Output length control	Summarize in exactly 3 sentences. Keep under 100 tokens.	• Specifies word/sentence/token count • a stated count is a soft target the model may miss; max_tokens is a hard truncation that can cut output mid-word and break JSON
Enumerated instructions	1. Extract entities 2. Classify sentiment 3. Return as table	• Numbered steps clarify sequence and expectations • improves task adherence when multiple operations are required
Format templates	<summary> <title>...</title> <body>...</body> </summary>	• Provides a markup skeleton for the model to fill • keeps nested or hierarchical output consistent; especially effective with XML
Preamble-free responses	Respond directly without preamble. Do not start with phrases like "Here is..." or "Based on...".	• A direct instruction that skips throat-clearing openers • replaces the older trick of prefilling the reply to force it past the intro
Negative prompting (constraints)	Do NOT include personal opinions. Avoid bullet points.	• Specifies what to exclude from output • unreliable on its own because models handle negation poorly; pair with positive framing (say what to include)

Technique	Example	Description
Relying on negative constraints	Instead of: "Don't use markdown." Try: "Write in flowing prose paragraphs."	- Counting how many of a system prompt's rules are phrased as negative constraints rather than positive instructions - relying on them as the primary way to steer behavior works worse than pairing each one with a positive instruction
Prompt-output style matching	Write the prompt itself in plain prose (no headers or bullets) to reduce markdown in the reply	- The formatting style of the prompt carries over into the response style - a fallback when explicit format instructions alone do not steer the output

TABLE 3

Message Roles and Context Structure

Chat models read a list of role-tagged messages rather than one block of text. The `system` (and newer `developer`) role sets standing behavior, `user` carries the live request, and `assistant` holds the model's prior replies. Because each request is stateless, your app resends the whole list every turn to maintain context, and higher-privilege roles outrank the user role when instructions conflict.

Concept	Example	Description
System message	You are a helpful assistant specializing in Python.	- Sets global behavior, persona, and constraints - applied before all user messages as persistent context, but it is guidance, not a security boundary
User message	How do I reverse a list in Python?	- Contains user query or command - the primary input the assistant responds to
Assistant message	<code>{"role": "assistant", "content": "Who's there?"}</code>	Model's previous reply , sent back as history so the conversation keeps its thread across turns
Developer message	<code>[developer] "Always respond in JSON format"</code>	- OpenAI's newer app-developer instruction role - ranks above user messages in the instruction hierarchy and is meant to win conflicts
Multi-turn context	<code>[user] "Define recursion"</code> <code>[assistant] "..."</code> <code>[user] "Give example"</code>	- Each request is stateless, so the client resends the full history every turn - longer chats cost more tokens and can exceed the context window
Long context prompting	<code><documents></code> <code><document index="1"></code> <code><source>report.pdf</source></code> <code><document_content>{{REPORT}}</document_content></code> <code></document></code> <code></documents></code>	- For 20k+ token inputs, place long documents above the query, instructions, and examples - wrap multiple documents in <code><document></code> tags with <code><source></code> and <code><document_content></code> subtags
Quote extraction	Find quotes relevant to the question. Place them in <code><quotes></code> tags, then answer using only those quotes.	Asking the model to quote the relevant passages first keeps a long-document answer grounded, instead of skimming the whole document
Turn-scoped reminders	<code>[system, turn 1] "Keep the reply under 3 sentences."</code> <code>[system, turn 12] "Keep the reply under 3 sentences."</code>	- Resends a short instruction (like a length limit) fresh each turn, so it does not fade out as the conversation grows - never edit or delete an earlier copy in place: that breaks the append-only history
Prefilled response (legacy)	<code>{"role": "assistant", "content": "Here is the requested summary:\n"}</code>	- A partial assistant message once used to force an output format or skip a preamble no longer supported on the last turn for current Claude models; use structured outputs instead

TABLE 4

Sample Selection and Example Design

Which examples go into a few-shot prompt, and in what order, often moves accuracy more than how many you add. The same handful of biases, similarity, label balance, and recency, explain why query-matched sets, contrastive pairs, and careful ordering all outperform picking examples at random.

Strategy	Example	Description
Random sampling	Pick 5 random examples from dataset	• Baseline approach • fast but mirrors data skew and ignores query relevance
Diverse examples	Typical: "Capital of France?" → Paris Edge case: "Capital of a country that no longer exists?" → explains the ambiguity Malformed: empty question → asks for clarification	Varies wording, length and edge cases across the demonstration set so the model learns the underlying task , not the surface pattern of one example
Example tags	<pre><examples> <example>Q: 2+2? A: 4</example> <example>Q: 3+5? A: 8</example> </examples></pre>	Wraps each demonstration in <code><example></code> tags (the set in <code><examples></code>) so the model reads them as samples to follow , not new instructions
Similarity-based selection	Choose examples most similar to query via embedding distance	• Provides contextually relevant demonstrations • often outperforms random, but similar examples cluster and can lose diversity
Stratified sampling	2 positive, 2 negative, 1 neutral sentiment	• Ensures balanced coverage of categories • counters majority-label bias when data is imbalanced
Contrastive examples	Correct: "Step A → B → C" Incorrect: "Step A → C (missing B)"	• Shows both correct and incorrect cases • helps the model see which reasoning steps to avoid
Example ordering	Place most relevant or recent examples last	• LLMs exhibit recency bias • reordering the same examples can swing accuracy from near chance to near best
Complexity-based prompting	Select few-shot examples with the most reasoning steps	• Prefers demonstrations with higher reasoning complexity (longer chains) • can also vote over the most complex chains at decoding; raises multi-step accuracy
Active-Prompt	Measure uncertainty on unlabeled questions → annotate most uncertain → add to few-shot pool	• Uses uncertainty sampling to select which examples a human should annotate • improves few-shot performance with minimal human labeling

TABLE 5

Generation Parameters and Sampling

These settings control how a model turns its next-token probabilities into actual text: how much randomness to allow, which low-probability tokens to discard, how long to keep going, and when to stop. Tuning them well is the difference between focused, parseable output and creative-but-unreliable rambling.

Parameter	Example	Description
Temperature	<pre>temperature=0.0 (deterministic) temperature=1.0 (creative)</pre>	• Controls randomness, not answer quality • lower = more focused/repetitive, higher = more diverse/creative • typical range 0-2; even 0 is not guaranteed bit-for-bit identical across runs
Top-p (nucleus sampling)	<pre>top_p=0.9</pre>	• Keeps the smallest token set whose cumulative probability $\geq p$, then samples from it • adapts the candidate count to the model's confidence • vendors recommend tuning temperature or top_p, not both

Parameter	Example	Description
Max tokens	<code>max_tokens=150</code>	• Hard cap on output length that truncates the moment it is hit • not a target length; can cut mid-sentence and break JSON • on newer chat APIs the parameter is deprecated in favor of <code>max_completion_tokens</code> and unsupported on reasoning models
Stop sequences	<code>stop=["###", "\n\n"]</code>	• Terminates generation when a specified string is produced (content-based, unlike the length-based max-tokens cap) • useful for structured outputs and preventing runaway text
Frequency penalty	<code>frequency_penalty=0.5</code>	Reduces repetition by penalizing tokens in proportion to how often they have already appeared (count-based)
Presence penalty	<code>presence_penalty=0.6</code>	Encourages topic diversity with a flat one-time penalty applied once a token has appeared at all, regardless of count
Top-k sampling	<code>top_k=40</code>	• Restricts sampling to the k most likely tokens (a fixed count) • simpler than top-p but a blunt cutoff that ignores the distribution's shape • many implementations default to 50 when left unset
Min-p sampling	<code>min_p=0.05</code>	• Keeps tokens above a fraction of the top token's probability (base value × top probability) • adaptive: strict when one token dominates, relaxed when the model is uncertain • pairs well with temperature > 1
Logit bias	<code>logit_bias={"50256": -100}</code>	• Directly boosts or bans specific tokens by adding a fixed value (-100 to 100) to their logits before sampling • -100 excludes a token, 100 makes it almost certain • a manual override, unlike the automatic frequency/presence penalties

TABLE 6

Emotion and Persona Techniques

These techniques shape how a model speaks and reasons by giving it a role, an audience, or an emotional frame. They mostly steer tone, depth, and perspective, and their effect on factual accuracy is far weaker and less reliable than popular advice suggests.

Technique	Example	Description
Specific persona	You are a food critic. You are a food critic writing for the Michelin Guide.	• A bare role label shifts tone only a little • adding real detail (publication, audience, credentials) is what makes a persona change the output further
Expert persona	You are a Pulitzer Prize-winning journalist. Write a headline.	• Assigns specific expertise or identity • mainly shapes tone, depth, and style, and does not reliably boost factual accuracy
Multi-persona prompting	Summon three experts (security, UX, backend). Have them collaborate on a review.	• One model simulates multiple expert personas collaborating in a single self-collaboration • produces more thorough, multi-perspective outputs
Emotional prompting	This is very important to my career. Please give your best answer.	• Adds emotional stakes or urgency • reported gains in earlier studies, but effects are mixed and model-dependent, often weaker on frontier models
Simulated Theory of Mind (SimToM)	Put yourself in the reader's shoes. What would they find confusing?	• Two-stage perspective-taking: filter context to what a character knows, then answer from that view • improves reasoning about beliefs and supports more empathetic responses

TABLE 7

Reasoning and Decomposition Techniques

These prompts get a model to work through a problem instead of guessing an answer in one leap. Some show step-by-step reasoning, others split a task into smaller parts, branch and backtrack across options, or interleave reasoning with real tool calls. Their gains depend heavily on model scale, and a written reasoning trace is not a guaranteed account of what the model actually did.

Method	Example	Description
Chain-of-Thought (CoT)	Q: $23 + 47 = ?$ A: $23 + 47 = 20 + 40 + 3 + 7 = 60 + 10 = 70$	- Prompts model to show step-by-step reasoning - an emergent ability that helps large models on math and logic, but can fail or hurt on small models
Zero-shot CoT	Let's think step by step.	Triggers reasoning without examples - effective shortcut when few-shot is impractical; redundant on reasoning models (o1/o3/R1)
Self-consistency	Generate 5 answers via CoT → select majority answer	- Samples multiple independent reasoning paths and takes the majority answer - improves reliability, but costs N times the tokens and latency
Universal Self-Consistency	Apply self-consistency to non-reasoning tasks (e.g., classification, extraction)	- Has the LLM itself pick the most consistent of several candidate answers - extends majority-voting benefits to free-form tasks where answers cannot be counted
Tree of Thoughts (ToT)	Evaluate 3 approaches → explore best 2 → backtrack if stuck	- Models reasoning as branching exploration with self-evaluation and backtracking - uses search over partial paths, handling planning and multi-path problems
Least-to-Most prompting	Step 1: Simplify equation Step 2: Solve for x using Step 1	- Decomposes a problem into ordered subproblems, each fed the previous answer - generalizes to problems harder than the examples shown
ReAct (Reasoning + Acting)	Thought: Need population data Action: search("France population") Observation: 67M → Answer	- Interleaves reasoning traces with tool actions and observations - grounds reasoning in retrieved results, reducing hallucination
Automatic Reasoning and Tool-use (ART)	New task → auto-selected demo from task library pause at tool call → resume with its output	Automatically pulls matching reasoning-and-tool-use demonstrations from a task library instead of hand-scripting them - pauses generation for a tool call and resumes with the result; humans can fix a demo or add a tool
Plan-and-Solve (PS+)	First, devise a plan to solve this. Then carry out the plan step by step.	- Model plans subtasks before executing them - a zero-shot method that reduces the missing-step and calculation errors of zero-shot CoT
Step-back prompting	Before answering, what general principles apply to this problem?	- Model identifies high-level concepts or first principles before specifics - improves reasoning on knowledge-intensive and abstract problems
Graph of Thoughts (GoT)	thought_1 + thought_2 → aggregated_insight Loop back for refinement	- Organizes reasoning as a directed graph that can merge branches and loop back - most flexible for complex interdependent reasoning
Thread of Thought (ThoT)	Walk me through this context step by step, summarizing as you go.	- Segments and analyzes long or chaotic contexts methodically - plug-and-play technique for tasks with extended or noisy input
Auto-CoT	Cluster questions by diversity → auto-generate CoT demos	Automatically constructs chain-of-thought demonstrations without manual effort - samples diverse questions so the occasional wrong auto-generated chain does little harm

Method	Example	Description
Self-Ask	Are follow-up questions needed? Yes: What is...? → intermediate answer Final answer: ...	- Model generates and answers sub-questions before the main answer - improves compositional and multi-hop reasoning

TABLE 8

Advanced Reasoning Patterns

These patterns push a model past a single answer by adding structure: writing its own prompts or examples, critiquing and verifying its work, offloading hard computation to code, or trading layout for speed. Knowing what each one actually changes (and where it quietly fails) is what separates a reliable pipeline from a fragile one.

Pattern	Example	Description
Meta-prompting	Generate a prompt to classify movie reviews.	- Model writes or optimizes prompts for a task - enables iterative self-improvement and automated prompt engineering
Generated knowledge prompting	First, list relevant facts about photosynthesis. Now answer: What role does chlorophyll play?	- Model generates intermediate knowledge before answering - improves factual accuracy on knowledge-intensive queries, using its own training (not external retrieval)
Self-Refine	Draft → Critique your draft → Revise based on feedback → repeat	- Model iteratively generates, critiques, and refines its own output, needing no external model - Lifts quality on generation tasks, but self-critique alone does not reliably fix reasoning errors
Chain of Verification (CoVe)	Answer → generate verification questions → answer each independently → revise	- Model plans verification questions, answers them independently of the draft, then revises - significantly reduces hallucinations in factual tasks
Program-Aided Language (PAL)	Write Python to solve: "If $x^2 = 16$, find x " def solve(): return sqrt(16)	- Model generates executable code as the reasoning step - offloads arithmetic to an interpreter that runs the code, for higher accuracy
Skeleton-of-Thought (SoT)	First: generate outline with 5 sections. Then: write each section in parallel.	- Creates structural outline first, then parallelizes content generation - reduces latency by up to 2.4x for long outputs
Reflexion	Review your answer. What could be improved? Revise → iterate	- Model self-critiques and writes a verbal reflection it stores in memory as context for the next attempt - reinforces the agent without any weight update or fine-tuning
Analogical prompting	Recall relevant problems similar to this, then solve by analogy.	- Model self-generates relevant examples before solving the task, eliminating manual few-shot curation - Improves math and code reasoning accuracy
Directional stimulus prompting	Keywords: protein, folding, disease Write an abstract.	- Provides hints or cues (keywords, themes) to steer generation toward desired content - a small trained policy model can generate the hints for a frozen LLM
Cumulative reasoning	Generate propositions iteratively → verify each → accumulate into final answer	- Uses a proposer, verifier, and reporter to build the answer from verified steps - verifying each proposition before accumulating it is what sets it apart from plain chain-of-thought
Chain of Density (CoD)	Summary 1: sparse (50 words) Summary 2: denser (same length, +3 entities) Iterate 5 times	Iteratively packs more entities into a fixed-length summary - produces human-preferred summaries by the later steps

Pattern	Example	Description
Maieutic prompting	Generate explanation tree → prune contradictory branches	- Builds an abductive, recursive tree of explanations - frames the answer as a satisfiability problem over their logical relations to find the most consistent one
ThoughtSculpt	Expand a thought tree with MCTS → evaluator scores each node → revise a weak branch instead of extending	- Searches a tree of candidate outputs with Monte Carlo Tree Search, an LLM evaluator scoring each node - Its action space includes revising a prior output instead of only extending it, which lifted scores across several reasoning tasks
Prompt repetition	What are the causes of inflation? What are the causes of inflation?	- Repeating the prompt twice gives a bidirectional-context effect in causal models, reported to help non-reasoning LLMs - doubles input token cost but adds no generated tokens or latency
DR-CoT (Dynamic Recursive CoT)	Recurse on sub-steps → truncate context → vote across reasoning chains	- Combines recursive refinement, dynamic context truncation within a token budget, and majority voting - helps parameter-efficient models rival larger ones, though voting can still fail under shared model bias

TABLE 9

Prompting for Reasoning Models

Reasoning models such as OpenAI's o-series, Claude with adaptive or extended thinking, and DeepSeek-R1 think before they answer, so they reward concise goal statements over heavy step-by-step scaffolding. These techniques cover how to steer and configure that hidden reasoning, keep it from running longer than a task needs, and avoid instructions older models needed but these models do not.

Technique	Example	Description
Reasoning vs GPT model choice	Reasoning model: "Reduce checkout drop-off." GPT model: "Change the CTA text to 'Buy Now' and enlarge the font."	- A reasoning model is like a senior co-worker: give it a goal and trust it to work out the details - a GPT model is like a junior co-worker: give it precise, explicit steps
Goal-oriented prompting	Solve for x where $3x + 7 = 22$. Show the solution process and final result.	- State desired outcome clearly without prescribing steps - reasoning models (o1/o3/R1) perform best with concise goal statements
Avoid explicit CoT instructions	Do not add "think step by step" to o1/o3/R1	- Reasoning models already reason internally - explicit CoT is redundant and can increase latency without benefit
Adaptive thinking	thinking={"type": "adaptive"} output_config={"effort": "high"}	- Claude decides on its own when and how much to think, based on effort and query complexity - the current default, replacing manual budget_tokens
Reasoning effort control	reasoning_effort: "high"	- Adjusts how deeply the same model reasons before answering "low" for simple tasks, "high" for complex problems; controls cost and latency
Interleaved thinking guidance	After receiving tool results, carefully reflect on their quality and determine optimal next steps before proceeding.	- A prompt line that tells the model to reason about a tool's result before its next action - steers thinking between tool calls, not just at the start
Commit-to-an-approach prompting	Choose an approach and commit to it. Avoid revisiting decisions unless new information contradicts your reasoning.	- Stops a model from re-exploring settled decisions, cutting thinking tokens and latency - add it when a model thinks longer than the task needs

Technique	Example	Description
Keeping reasoning costs low	<code>max_output_tokens: 2000</code>	• Caps total generated tokens, including hidden reasoning tokens, to bound spend • pair with a lower effort setting for routine tasks
Extended thinking (budget tokens) (legacy)	<code>thinking: {type: "enabled", budget_tokens: 10000}</code>	• Allocates a reasoning scratchpad via a fixed <code>budget_tokens</code> cap, drawn from <code>max_tokens</code> • deprecated: Claude 4.7+ rejects it; use adaptive thinking with <code>effort</code> instead

TABLE 10

Prompt Chaining and Workflow Orchestration

Once a task is too big for one prompt, you compose several model calls into a pipeline. These patterns range from simple sequential chains to retrieval, tool use, routing, and self-directing agents. A key theme: the model proposes structured steps, but your code executes tools, routes branches, and enforces stop conditions.

Technique	Example	Description
Prompt chaining	Prompt 1: Extract entities → <code>output_1</code> Prompt 2: Classify entities from <code>{output_1}</code>	• Decomposes a task into sequential LLM calls • each prompt's output feeds the next, so steps stay simple and easy to debug
Retrieval-Augmented Generation (RAG)	1. Retrieve docs about "mitochondria" 2. Prompt: "Using {docs}, explain ATP synthesis"	• Fetches external documents at query time and adds them to the prompt • grounds answers in current or proprietary data without retraining the model
Function calling (tool use)	<code>tools: [{"name": "get_weather", "parameters": {"location": "string"}}]</code>	• Model selects a structured tool schema and emits name plus arguments • your application code runs the tool, so validate arguments before executing
Conditional branching (routing)	If sentiment=negative: call <code>escalation_prompt</code> Else: call <code>thank_you_prompt</code>	• Classifies the input, then routes it to a specialized prompt • separates concerns so each branch stays focused on one kind of case
Parallelization	Sectioning: <code>content_call + guardrail_call</code> (parallel) Voting: <code>same_prompt x3</code> → majority vote	• Runs several LLM calls at once and aggregates the outputs programmatically • sectioning splits one task into independent parts, voting repeats the same task for diverse takes
Orchestrator-workers	Orchestrator: plan subtasks → delegate to <code>worker_1, worker_2</code> Orchestrator: merge worker results	• A central LLM plans subtasks it could not predict in advance and delegates each to a worker call • more flexible than parallelization, whose subtasks are fixed upfront
Evaluator-optimizer	Generator: draft translation Evaluator: critique vs criteria → Generator: refine	• One call drafts, a second call critiques it against criteria and the loop repeats until it passes • works best when feedback measurably improves the draft
Agentic workflows	Agent: Plan → Act → Observe → Refine → Act	• Model directs its own steps, choosing tools based on each result • loops toward a goal, so a max-iteration cap is needed to avoid runaway cost
ReWOO (Reasoning Without Observation)	Plan all tool calls upfront → execute → synthesize	• Decouples planning from observation • a planner writes the full plan with placeholders, workers run tools, a solver combines results, cutting LLM calls vs ReAct

TABLE 11

Prompting Agents and Long-Horizon Tasks

Wording that steers a tool-using agent once it starts acting: whether it takes action or only suggests, how hard it reaches for a tool or a subagent, and how it keeps its place across a long task or a fresh context window. These instructions belong in the system prompt itself, separate from the architecture that chains the calls together.

Pattern	Example	Description
Default to action	<code><default_to_action></code> By default, implement changes rather than only suggesting them. If the user's intent is unclear, infer the most useful likely action and proceed. <code></default_to_action></code>	Tells the model to implement a change instead of only suggesting one when a request like "can you improve this" implies action.
Tool-triggering calibration	CRITICAL: You MUST use this tool when... Use this tool when...	Softens forceful tool-forcing language that once fixed undertriggering but now makes newer models overtrigger a tool.
Detailed tool descriptions	Deleting tasks: Use the delete_task tool to clear up all the tasks before starting the search plan.	Explains when and why to use a tool in the system prompt itself , not only in the tool's parameter spec.
Parallel tool calling	<code><use_parallel_tool_calls></code> If you intend to call multiple tools and there are no dependencies between the tool calls, make all of the independent tool calls in parallel. <code></use_parallel_tool_calls></code>	Fires independent tool calls simultaneously instead of one after another, but never for calls that depend on each other's output.
Avoid over-engineering	Avoid over-engineering. Only make changes that are directly requested or clearly necessary.	Caps unrequested scope creep: no extra abstractions, cleanup or defensive code beyond what was asked .
Avoid hard-coding to pass tests	Do not hard-code values or create solutions that only work for specific test inputs.	Blocks shortcuts that only satisfy the given test cases instead of solving the problem generally .
Investigate before answering	<code><investigate_before_answering></code> Never speculate about code you have not opened. If the user references a specific file, you MUST read the file before answering. <code></investigate_before_answering></code>	Requires opening a file before making any claim about it, cutting hallucinated answers about a codebase.
Balancing autonomy and safety	Destructive: deleting files or branches, <code>rm -rf</code> Hard to reverse: <code>git push --force</code> , <code>git reset --hard</code> Visible to others: pushing code, sending messages	Has the model weigh reversibility before acting: local edits proceed, destructive or shared-system actions ask first.
Long-horizon state tracking	<code>// progress.txt</code> Session 3 progress: - Fixed authentication token validation - Next: investigate user_management test failures	Tracks progress with a structured status file, freeform notes or git commits so a task survives across sessions .
Context awareness across windows	Your context window will be automatically compacted as it approaches its limit, allowing you to continue working indefinitely from where you left off.	Lets the model track its own remaining token budget and keep working through a compaction instead of stopping early .
Subagent orchestration damping	Use subagents when tasks can run in parallel, require isolated context, or involve independent workstreams. For single-file edits, work directly.	Reins in automatic subagent delegation to cases with parallel or isolated work, not simple single-file tasks .

TABLE 12

Multimodal and Vision-Language Prompting

Multimodal prompting feeds a model more than text. You pass an image or audio clip alongside your question, and the model reasons over both. Keep in mind these models do not "see" or "hear" perfectly: they give approximate object counts, struggle with precise spatial detail, and can hallucinate text when reading documents, so verify anything high-stakes.

Approach	Example	Description
Image + text prompting	[image of chart] What trend does this show?	- Combines visual and textual input in one message - model analyzes image content to answer the text query
Image-first ordering	content: [{type: "image", ...}, {type: "text", text: "Summarize the findings."}]	Placing the image or scanned document before the question in the same message improves accuracy, the same rule long text documents follow
Visual question answering	[image of room] How many chairs are visible?	- Model performs object counting, detection, or scene understanding from an image - Counts are approximate, so verify them
OCR and document understanding	[scanned receipt] Extract total amount.	- Reads and interprets text within images, including tables, forms, and structured documents - May hallucinate plausible but wrong values
Image captioning	[photo] Generate detailed caption.	Model produces a natural language description of image content
Visual reasoning	[two images] Which object is larger?	- Requires comparison or relational reasoning across visual inputs - Precise spatial localization is unreliable
Multi-image labeling	"Image 1:" [chart A] "Image 2:" [chart B] Compare growth in Image 2 vs Image 1.	Introducing each image with a short label (<code>Image 1:</code> , <code>Image 2:</code>) before its content lets you refer back to a specific image later in the same prompt or conversation
Crop-and-zoom tool	tool: crop_image(x, y, w, h) Zoom into the legend and read the values.	Giving the model a tool that crops and magnifies part of the image boosts accuracy on charts, documents, and diagrams with fine detail
Audio prompting	[audio clip] Transcribe and summarize this meeting.	- Processes speech or audio input natively alongside text - supported by audio-capable chat models for transcription, analysis, and translation

TABLE 13

Domain-Specific Applications

Prompt engineering plays out differently across common LLM tasks. Code and data work reward low temperature and strict structure, classification and summarization lean on fixed label sets and techniques like Chain of Density, and grounded question answering depends on retrieval. Knowing the right tool and setting per task is what separates a reliable pipeline from a flaky one.

Domain	Example	Description
Code generation	Write a Python function to merge two sorted lists.	- Produces runnable code, but output can look correct yet miss details like an import - favor low temperature and always test the result
Data extraction	Extract: name, email, phone from: "Contact John at john@ex.com"	- Pulls structured fields from unstructured text - structured outputs enforce a JSON schema at decode time, far more reliable than free-text or plain JSON mode

Domain	Example	Description
Classification	<pre>from enum import Enum class Category(str, Enum): violence = "violence" sexual = "sexual" self_harm = "self_harm"</pre>	- Restricts the answer to one label from a small, fixed set instead of open text - common for routing, tagging and moderation, where only a few valid categories exist
Sentiment analysis	Classify sentiment: "I loved this movie!" → positive	- Determines the emotional tone of a piece of text - few-shot examples can steer the model to match your own labeling scheme
Summarization	Summarize this article in 2 sentences.	- Condenses long text into key points - Chain of Density packs more entities into a fixed length and reduces lead bias
Question answering	Based on: {document}, answer: Who founded the company?	- Provides a factual answer from context - RAG grounds answers in private or fresh sources, but is unneeded for general knowledge
Creative writing	Write a haiku about autumn.	- Generates poetry, stories, or dialogue - higher temperature adds variety, but too high turns coherent text into nonsense
Translation	Translate to German: "Good morning"	- Converts text between languages - few-shot in-context examples improve translation quality with no fine-tuning needed

TABLE 14

Optimization and Automation

These methods move prompt work from hand-tuning to measured, repeatable engineering: define success criteria and evals before you touch a prompt, then compare, test, and version what you ship (A/B testing, repeat-run reliability, prompt versioning). Prompt caching cuts cost and latency on a static prefix, while APE, DSPy, and soft prompts push prompt creation itself onto a model or a training process.

Method	Example	Description
Define success criteria and evals	Target: F1 of at least 0.85 on a held-out test set → grade with exact match, code, or an LLM grader	- Set specific, measurable pass/fail criteria and a test set before writing or changing a prompt - Automate the grading so prompt changes are compared objectively, not by eyeballing outputs
A/B testing prompts	Run variant A vs B on the same inputs → measure accuracy, latency, cost → deploy winner	- Empirical comparison to select the best prompt for production - needs enough samples for statistical confidence, since LLM outputs vary
Repeat-run reliability testing	<code>promptfoo eval --repeat 5</code> → same test run 5 times → outputs compared for consistency, not just correctness	- Checks whether one prompt's own output drifts run to run, apart from comparing it against a different prompt - Surfaces a format or answer that only sometimes holds, which a single run would never show
Prompt versioning	Track each prompt change as an immutable, identified version with eval metrics	- Manages prompt iterations in production - enables exact rollback, A/B testing, and regression tracking
Prompt caching	Place static system instructions first → variable content last	- Providers reuse the computed prefix for an exact match, cutting cost up to ~90% and latency up to ~80% - caches the input prefix, never the response; supported by OpenAI, Anthropic, Google
Automatic Prompt Engineering (APE)	Generate prompt candidates → score on a dataset → select best performer	- A model proposes instruction candidates, which are then scored and filtered on a validation set - replaces manual trial-and-error
DSPy framework	Define signatures → framework compiles and optimizes prompts from examples and a metric	- Declarative approach where prompts are compiled, then iteratively improved against a metric, not hand-written - discards variations that do not score better

Method	Example	Description
Prompt tuning (soft prompts)	Learn a few continuous embedding vectors prepended to the input while the model stays frozen	- Trains small learnable vectors, not readable text, leaving model weights frozen - parameter-efficient alternative to full fine-tuning

TABLE 15

Safety and Robustness

Securing an LLM application means assuming its prompts and the data it reads are adversarial: keep untrusted input from overriding your instructions, keep the system prompt itself from leaking, and validate what the model emits before it reaches code or a user. The reverse failure matters just as much: a harmless request that a cautious safety layer wrongly blocks.

Technique	Example	Description
Prompt injection defense	Use input handling , instruction delimiters, and privilege limits	- Mitigates attacks where input tries to override developer instructions or exfiltrate data - OWASP ranks prompt injection as LLM01, the top LLM risk
Indirect prompt injection defense	Separate trusted instructions from untrusted external data using privilege boundaries	- Prevents attackers from embedding hidden instructions in documents, emails, or tool outputs the model processes - The key risk for agentic and RAG systems
Output validation	Check output against a schema, encode it, or screen with a secondary LLM	Treats model output as untrusted before it reaches a browser, database, or shell, preventing XSS, SQL injection, or command execution
Reduce prompt leak	Isolate secrets in the system prompt , away from user turns → filter output for leaked keywords → audit regularly	- Cuts the risk that a user extracts the hidden system prompt, mainly by separating context from queries and filtering output - Adds complexity that can hurt task performance, so use it only when a leak would matter
Jailbreak resistance	Detect attempts to bypass safety via role-play, encoding, or indirection	Models trained to recognize and refuse disguised harmful requests , targeting the model's safety rules (distinct from injection)
Red-teaming prompts	Run adversarial probes such as injection and jailbreak attempts before launch	Adversarial testing to find vulnerabilities before attackers do . Expected by the NIST AI RMF and OWASP LLM Top 10
Constitutional AI principles	Model self-critiques against rules like Refuse harmful requests. Be helpful and honest.	A training method (RLAIF): the model critiques and revises its own answers against a set of principles, not a runtime word filter
Reframe to avoid over-refusal	Ask for shoplifting deterrence tips instead of how not to get caught shoplifting → same topic, compliant framing	- The same underlying need, worded around the legitimate use instead of the exploit, turns a refusal into a helpful answer - Add the context a cautious classifier is missing (who's asking, why) instead of repeating the same wording

TABLE 16

Anti-Patterns and Common Pitfalls

These are the prompt habits that quietly wreck output quality, run up cost, or produce confidently wrong answers. For each one, the fix is usually the opposite move: be specific, decompose, show examples, set limits, mark boundaries, ground recent facts with retrieval, and match sampling to the task. The last row is an evolving guideline, heavy step-by-step scaffolding helps weaker models but can over-constrain frontier ones.

Pattern	Example	Description
Vague instructions	Tell me about AI.	- Lacks specificity - produces generic, unfocused output; always specify scope, audience, or format
Overloading single prompt	Mixing 10 unrelated tasks in one prompt	- Splits the model's attention, so every task gets shallow output - better to chain or decompose into separate prompts

Pattern	Example	Description
No examples for complex tasks	Zero-shot on nuanced classification	• Underperforms without demonstrations • 2-5 few-shot examples teach your exact criteria; more than that tends to plateau
Ignoring output length	No length constraint, leading to a 5000-word response	• Generates unnecessarily long outputs and runs up cost at scale • state the length in words; max-tokens only truncates, it does not shape length
Ambiguous delimiters	Input: text here Output: more text (no clear boundary)	• Model confuses what to process vs. generate • wrap each part in its own <tag> (<instructions> , <context>) instead of plain text boundaries
Assuming knowledge cutoff awareness	What happened last week? (model trained months ago)	• Model cannot access real-time data and will confidently invent recent facts • ground it with RAG or tool use
Wrong parameters for task	Deterministic task with temperature=1.5	• Excessive randomness where consistency is needed (temperature is not truthfulness) • tune temperature and top-p to the task
Anti-laziness prompting	"CRITICAL: You MUST use this tool when..."	• Aggressive tool-triggering language written for older, less proactive models makes newer models overtrigger • retest old system prompts against the current model and dial back to plain phrasing ("Use this tool when...")
Excessive scaffolding for capable models	10-step procedural instructions for a frontier model on an open-ended task	• Over-constraining can hinder autonomous reasoning in frontier models • describe the desired result and let it choose the route; strict steps still suit procedural, schema-bound tasks

References

Official Documentation

- Prompt elements
- Zero-shot prompting
- Few-shot prompting
- One-shot prompting
- Role prompting
- Instruction following
- Structured output (JSON)
- XML tag structuring
- Delimiters and sections
- System message
- User message
- Assistant message
- Turn-scoped reminders
- Contrastive examples
- Temperature
- Top-k sampling
- Logit bias
- Emotional prompting
- Zero-shot CoT
- Generated knowledge prompting
- Program-Aided Language (PAL)
- Analogical prompting
- Directional stimulus prompting
- Cumulative reasoning
- Reasoning vs GPT model choice
- Reasoning effort control
- Extended thinking (budget tokens) (legacy)
- Prompt chaining
- Function calling (tool use)
- Detailed tool descriptions
- Image + text prompting
- Image-first ordering
- Visual reasoning
- Audio prompting
- Code generation
- Data extraction
- Sentiment analysis
- Question answering
- Creative writing
- Define success criteria and evals
- A/B testing prompts
- Repeat-run reliability testing
- Prompt caching
- Prompt tuning (soft prompts)
- Reduce prompt leak
- Red-teaming prompts
- Overloading single prompt
- Ignoring output length
- Assuming knowledge cutoff awareness
- Instruction following
- Max tokens
- Sentiment analysis
- Audio prompting - <https://platform.openai.com/docs/guides/audio>
- Top-k sampling - https://huggingface.co/docs/transformers/generation_strategies
- Vague instructions - <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>
- Visual question answering - <https://platform.openai.com/docs/guides/vision>

57. Creative writing - <https://www.promptingguide.ai/techniques>

58. Question answering - <https://www.promptingguide.ai/techniques/rag>

Specifications & Standards

59. Prompt injection defense

60. Output validation

Academic Papers

61. Rephrase and Respond (RaR)

63. Stratified sampling

65. Complexity-based prompting

67. Expert persona

69. Simulated Theory of Mind (SimToM)

71. Self-consistency

73. Tree of Thoughts (ToT)

75. ReAct (Reasoning + Acting)

77. Plan-and-Solve (PS+)

79. Graph of Thoughts (GoT)

81. Auto-CoT

83. Self-Refine

85. Skeleton-of-Thought (SoT)

87. Chain of Density (CoD)

89. ThoughtSculpt

91. DR-CoT (Dynamic Recursive CoT)

93. Translation

62. Random sampling

64. Example ordering

66. Active-Prompt

68. Multi-persona prompting

70. Chain-of-Thought (CoT)

72. Universal Self-Consistency

74. Least-to-Most prompting

76. Automatic Reasoning and Tool-use (ART)

78. Step-back prompting

80. Thread of Thought (ThoT)

82. Self-Ask

84. Chain of Verification (CoVe)

86. Reflexion

88. Maieutic prompting

90. Prompt repetition

92. ReWOO (Reasoning Without Observation)

94. Automatic Prompt Engineering (APE)

Articles & Tutorials

95. Contextual prompting

97. Min-p sampling

99. Retrieval-Augmented Generation (RAG)

101. Image captioning

103. Prompt versioning

105. Jailbreak resistance

107. Reframe to avoid over-refusal

109. Indirect prompt injection defense -
<https://www.anthropic.com/research/prompt-injection-defenses>

111. OCR and document understanding -
<https://www.datacamp.com/blog/top-vision-language-models>

96. Negative prompting (constraints)

98. Meta-prompting

100. Conditional branching (routing)

102. Summarization

104. DSPy framework

106. Constitutional AI principles

108. Agentic workflows -
<https://www.anthropic.com/research/building-effective-agents>

110. Excessive scaffolding for capable models - <https://www.mindstudio.ai/blog/how-to-prompt-gpt-5-5-outcome-first-prompting>

112. Stratified sampling -
<https://www.prompthub.us/blog/the-few-shot-prompting-guide>